

DNNF90 Specification

High-order partial-derivative propagation through neural networks

by multi-index partial Bell polynomials — v1.4

August 2026

Contents

1	Overview	1
1.1	Purpose and position	1
1.2	Scope	1
2	Mathematical specification	1
2.1	Notation	1
2.2	Forward propagation (high order)	2
2.3	High-order derivatives of tanh	2
2.4	Reverse propagation (adjoint)	2
2.5	Closure sets	2
3	Software structure	2
3.1	The layer boundary: mechanism and policy	2
3.2	File layout	2
3.3	The synchronization invariant	3
3.4	Uniqueness of the propagation engine	3
4	Library API	4
4.1	The contract	4
4.2	<code>multi_index_bell_module</code> : multi-indices and tables	4
4.3	<code>net_module</code> : networks and evaluation	4
4.4	<code>train_module</code> : training	4
4.5	<code>kalman_module</code> : extended-Kalman-filter training	5
4.6	<code>committee_module</code> : uncertainty by committee	5
4.7	<code>blas_wrap_module</code>	5
4.8	The C bindings	6
5	Input specification (<code>input_nn.dat</code>)	6
5.1	Format conventions	6
5.2	Required keys and defaults	6
5.3	Network and training	7
5.4	Choosing the activation	7
5.5	Evaluating a high-order fit	8
5.6	Loss terms and their data files	8
5.6.1	The form of the loss	8
5.6.2	What each form requires	9
5.6.3	Worked examples	9
5.6.4	The carried multi-index set and N_α	10
5.6.5	Key list	11
5.7	Residuals (collocation)	11
5.8	Coupled systems (the <code>System</code> block)	12
5.9	Uncertainty by committee	13
5.10	Optimizers	14

6	Output files	16
6.1	The training history <code>history_ep<start epoch, 7 digits>.dat</code>	16
6.2	Predictions	17
6.3	Weights and restarting	17
6.4	Reference information	17
7	Verification specification	17
8	Application to force fields	18
8.1	Verification	18
8.2	The smoothness requirement (important)	19
8.3	Result	19
9	Shipped examples and test cases	19
10	Cost and memory	19
11	Known limitations	20

1 Overview

1.1 Purpose and position

DNNF90 is a Fortran 90/95 library that constructs the output of a fully connected neural network together with its **mixed partial derivatives of arbitrary order** in a single forward pass, and returns the exact weight gradient of any loss built from those derivatives in a single reverse pass. The core is the recursion for high-order derivatives of composite functions organized by multi-index partial Bell polynomials; no nested automatic differentiation and no finite differences are used.

The software is released under the MIT license. It has two layers.

- **The library** (`lib/`): the embeddable mechanism. One-point evaluation, one-point seeded gradients, one optimizer step applied to a caller-owned array, committee estimation, Kalman-filter training, and the C bindings.
- **The training application** (`app/`): the keyword-driven reference implementation. Minibatches, validation, early stopping, eleven optimizers, file I/O, MPI.

The boundary follows the separation of mechanism from policy (Section 3.1).

1.2 Scope

- The hidden activation is tanh only, because the derivative tables are built from the Bell polynomials of tanh.
- The output layer is linear (identity) with a single output; the propagated derivatives are those of a linear read-out of the last hidden layer.
- The input dimension D_0 and the maximum order K are given in the input. The carried multi-index set is either dense (all $|\alpha| \leq K$) or the downward closure of a prescribed seed set.
- The loss lives outside the library. The library receives $\partial L / \partial T_\alpha$ (the *seed* below) and returns $\partial L / \partial W$.

2 Mathematical specification

2.1 Notation

Layers $l = 1, \dots, L$, width n_l of layer l , weights $W_{ji}^{(l)}$ ($i = 0$ is the bias). Input $\mathbf{x} \in \mathbb{R}^{D_0}$. Multi-index $\alpha = (\alpha_1, \dots, \alpha_{D_0})$, $|\alpha| = \sum_i \alpha_i$. The carried quantities are

$$T_j^{(l, \alpha)} = \frac{\partial^{|\alpha|} z_j^{(l)}}{\partial x_1^{\alpha_1} \dots \partial x_{D_0}^{\alpha_{D_0}}}, \quad S_j^{(l, \alpha)} = \frac{\partial^{|\alpha|} a_j^{(l)}}{\partial \mathbf{x}^\alpha},$$

where $a^{(l)} = W^{(l)} z^{(l-1)}$ and $z^{(l)} = \sigma(a^{(l)})$ ($\sigma = \tanh$; the last layer is the identity).

2.2 Forward propagation (high order)

The linear part is linear multi-index by multi-index:

$$S_j^{(l,\alpha)} = \sum_i W_{ji}^{(l)} T_i^{(l-1,\alpha)}, \quad T_0^{(0)} \equiv 1 \text{ (the bias channel)}.$$

The nonlinear part is the high-order derivative of a composition, expressed through the multi-index partial Bell polynomials $B_{\alpha,q}$:

$$T_j^{(l,\alpha)} = \sum_{q=1}^{|\alpha|} \sigma^{(q)}(S_j^{(l,0)}) B_{\alpha,q}(S_j^{(l,\cdot)}),$$

where $B_{\alpha,q}$ is a polynomial in the lower multi-index components of S , built by the recursion $B_{\alpha,q} = \sum_{\beta} c_{\alpha\beta} S^{(\beta)} B_{\alpha-\beta,q-1}$. The coefficients c and the referenced indices are tabulated once at initialization. The input layer carries $T_j^{(1,0)} = x_j$, $T_j^{(1,e_j)} = 1$, and zero otherwise.

2.3 High-order derivatives of tanh

$\sigma^{(q)}$ closes as a polynomial in $t = \tanh$: $\sigma^{(0)} = t$ and $\sigma^{(q+1)} = (1 - t^2) d\sigma^{(q)}/dt$, carried as a recursion on coefficient arrays, so that $q = 0, \dots, K + 1$ costs one polynomial evaluation per point.

2.4 Reverse propagation (adjoint)

Let the seed of a loss L be $\bar{T}_1^{(L,\alpha)} = \partial L / \partial T_1^{(L,\alpha)}$. The adjoint quantities are

$$\bar{S}_j^{(l,\beta)} = \sum_{\alpha} \bar{T}_j^{(l,\alpha)} h_{j,\alpha\beta}^{(l)}, \quad \bar{T}_i^{(l-1,\alpha)} = \sum_j W_{ji}^{(l)} \bar{S}_j^{(l,\alpha)},$$

$$\frac{\partial L}{\partial W_{ji}^{(l)}} = \sum_{\alpha} \bar{S}_j^{(l,\alpha)} T_i^{(l-1,\alpha)}.$$

h is assembled from the $\sigma^{(q+1)}$ values saved in the forward pass paired with the Bell tables. The gradient is accumulated in a fused per-layer sweep as soon as $\bar{S}^{(l)}$ is complete, so $\bar{T}, \bar{S}$ hold only one layer's plane at a time.

2.5 Closure sets

Given a seed multi-index set $\mathcal{A}$, the set actually carried is its **downward closure** $\{\beta \mid \exists \alpha \in \mathcal{A}, \beta \leq \alpha\}$. A non-closed set does not close the recursion. The dense set $|\alpha| \leq K$ is a special case of a closure. Indexing is lexicographic order with binary search (`find_alpha`), which remains viable at fifty to a hundred descriptors.

3 Software structure

3.1 The layer boundary: mechanism and policy

The library provides **mechanism**: evaluate one point, form one point's weight gradient from a seed, apply one step to a caller-owned array. The trainer holds **policy**: batch size, validation cadence, stopping rules, file output.

The boundary is forced by the embedding cases. Density-functional and molecular-dynamics host codes own their outer loop, their data flow and their parallel decomposition, so a library that imposes an epoch loop, a validation split or a checkpointing policy cannot be embedded. Mechanism can stay pure and reentrant (one work space per thread is the only requirement) and admits exact verification by finite differences; no comparable check exists for a stopping rule. The optimizer asymmetry (two rules in `lib/`, eleven in `app/`) follows from the same principle. The relationship mirrors that of PyTorch proper to Lightning; `app/` ships as the library's largest worked example.

3.2 File layout

File	Lines	Role
lib/ (the embeddable library)		
<code>multi_index_bell_module</code>	677	multi-index generation, downward closure, Bell tables, high-order tanh derivatives, lexicographic indexing, table sets (<code>tabset_t</code>)
<code>net_module</code>	322	create/read/write/free of <code>net_t</code> , evaluation work space, <code>net_eval_hod</code>
<code>train_module</code>	400	<code>net_forward_point</code> / <code>net_backward_point</code> / <code>net_grad_point</code> , the batch gradient <code>net_grad_batch</code> , gradient accumulators, <code>opt_sgd_step</code> / <code>opt_adam_step</code>
<code>committee_module</code>	105	uncertainty estimation by a committee of networks
<code>kalman_module</code>	239	per-pattern extended-Kalman-filter training
<code>c_api_module</code> + <code>dnnf90.h</code>	479	the C bindings (22 functions)
app/ (the keyword-driven trainer)		
<code>global_variables</code>	—	shared state
<code>io_module</code>	—	input parsing, data reading, output formats
<code>lib_net_module</code>	210	the lib network the trainer holds; weight synchronization and the entry points
<code>pinn_module</code>	59	residual evaluation and residual seeds (policy)
<code>init_weight_module</code>	—	eight initializations
<code>optimizer_module</code>	—	eleven optimizers
<code>sgd_batch_module</code>	—	epoch loop, minibatches, early stopping, MPI averaging
<code>validation_module</code>	—	validation costs, RMSE/MAE, output
<code>hod_check_module</code>	—	in-run finite-difference self-check
<code>api_module</code>	—	Fortran API onto the trainer state

3.3 The synchronization invariant

Because the trainer owns the master weight array, the invariant “the library network must hold the current weights” arises. It is enforced mechanically, not by discipline: every write to `weight` and `weight_best` advances a generation counter, each synchronization records the generation and the source array, and every evaluation entry point compares the two before computing and stops on a mismatch. The check is one integer comparison and stays enabled in production builds, because a missed synchronization returns *plausible numbers computed from the previous weights* — the most dangerous failure mode this layer has.

3.4 Uniqueness of the propagation engine

There is **exactly one** propagation engine in the tree. Training, the validation cost, the prediction output, the self-check and the C API all pass through `net_forward_point` / `net_backward_point` / `net_grad_batch`. The trainer keeps its own weight array as the master copy (checkpoints, restarts, MPI averaging and the optimizer states are expressed in it); `lnet_sync_weights` copies it into the library network once per batch iteration and at every point where the weights can have moved (just before validation, at output writing, inside the finite-difference loops of the self-check).

The entry points of `app/lib_net_module.f90`:

Entry	Meaning
<code>lnet_sync_weights(w)</code>	copy the trainer’s weights into the library network
<code>lnet_forward_value(x,v)</code>	one point’s value
<code>lnet_forward_hod(x,t)</code>	one point’s value and all carried derivatives, into the caller’s array <code>t</code>
<code>lnet_nalpha()</code>	the number of carried multi-indices (so the caller can size arrays)
<code>lnet_value_grad(x,y,c,∇)</code>	accumulate one point’s value-fit weight gradient
<code>lnet_seed_grad(seed,∇)</code>	accumulate the adjoint of an arbitrary seed
<code>lnet_seed_row(seed,row)</code>	write the adjoint of an arbitrary seed as one row (the natural-gradient metric)
<code>lnet_value_row(row)</code>	the row of a unit seed on the value component
<code>lnet_batch_grad(X,Y,c,∇)</code>	the value-fit gradient of a whole minibatch at once

4 Library API

`lib/` is the embeddable mechanism and is usable independently of the training application; this section describes everything it exposes. `make lib` builds the static library (`build/libdnnf90.a`); `make shared` the shared one. From Fortran, `.mod` files are compiler- and version-specific, so embeddings that cross toolchains should use the C bindings (Section 4.8).

4.1 The contract

The training contract is per point:

$$\text{net_grad_point} : \underbrace{\text{seed}_\alpha = \frac{\partial L}{\partial T^\alpha}}_{\text{assembled by the caller}} \mapsto \frac{\partial L}{\partial W}$$

The content of the loss L , the chain rule from descriptors to inputs, and the meaning of grids or collocation points all belong to the caller; the library does not know them. There is no shared state, and one work space per thread suffices for concurrent calls (the library is built with `-frecursive`).

4.2 `multi_index_bell_module`: multi-indices and tables

Element	Role
<code>init_hod_tables(D0,K,ns,seeds)</code>	build the shared tables. <code>ns=0</code> : the dense set ($ \alpha \leq K$); <code>ns>0</code> : the downward closure of the seed set
<code>tabset_t</code>	a table-set type; several networks can carry different multi-index sets at once
<code>tabset_init</code> , <code>tabset_from_current</code> , <code>tabset_free</code>	construct a table set (fresh, or copied from the shared tables) and free it
<code>alpha_index(alpha)</code>	slot number of a multi-index (lexicographic order + binary search)
<code>alpha_list</code> , <code>alpha_deg</code> , <code>ind.e1</code>	the list of multi-indices, their orders, the slots of the unit multi-indices e_i
<code>NUM_alpha</code> , <code>HOD_D0</code> , <code>HOD_KMAX</code>	the carried count and the D_0 , K of construction
<code>hod_tables_ready</code>	whether the shared tables are built
<code>tanh_derivs_hod</code> , <code>tanh_derivs_ts</code>	$\tanh^{(q)}$ for $q = 0..K+1$ by recursion
<code>write_alpha_order(file)</code>	write the multi-index ordering to a file
<code>bell_*</code> , <code>pair_*</code> , <code>fq_*</code>	the Bell, adjoint and forward coefficient tables (internal structure, but public)

4.3 `net_module`: networks and evaluation

Element	Role
<code>net_t</code>	the network type (layer count, widths, weights, a reference to its table set)
<code>net_init(nt,L,ndim,ts)</code>	construction; <code>ts</code> decides the carried multi-index set
<code>net_load</code> , <code>net_save</code>	read/write in the <code>nn_weight.dat</code> format
<code>net_free</code>	release; derived types are not auto-released through <code>intent(OUT)</code> , so always call it
<code>work_t</code> , <code>work_init</code> , <code>work_free</code>	the evaluation work space, one per thread
<code>net_eval_hod(nt,wk,x,t)</code>	one point's value and all carried derivatives (the inference path)

4.4 `train_module`: training

Element	Role
<code>twork_t</code> , <code>twork_init</code> , <code>twork_free</code>	the adjoint work space (holds the forward pass's intermediates)

Element	Role
<code>grad_t, grad_init, grad_free</code> <code>grad_zero, grad_add</code> <code>net_forward_point(nt,tw,x,t)</code>	the gradient accumulator; optimizer moments live here too zeroing and addition; <code>grad.add</code> is the thread-parallel reduction one point's forward pass; returns all carried derivatives and keeps the adjoint intermediates
<code>net_backward_point(nt,tw,seed,g)</code> <code>net_grad_point(nt,tw,x,seed,g)</code>	the adjoint of a seed, accumulated into <code>g</code> the two above in one call
<code>bwork_t, bwork_init, bwork_free</code> <code>net_grad_batch(nt,bw,X,Y,c,g)</code> <code>opt_sgd_step, opt_adam_step</code>	the batch work space (carries a batch axis) the batch gradient of a value fit; three matrix products per layer one step applied to a caller-owned array

4.5 kalman_module: extended-Kalman-filter training

A training route separate from gradient descent, used directly from the library or a host code (the application's Kalman option in `GD_method` is the trainer-side counterpart). It is the method in standard use for high-dimensional neural-network potentials of the n2p2 family.

One update presents one scalar observable (an energy, or one force component the host has chained into descriptor space) and applies the rank-one form; no matrix is inverted:

$$\begin{aligned}\xi &= y_{\text{obs}} - \hat{y}, & Pj &= Pj, & K &= \frac{Pj}{\lambda + j^\top Pj}, \\ w &\leftarrow w + K\xi, & P &\leftarrow \frac{P - K(Pj)^\top}{\lambda}, & \lambda &\leftarrow \lambda_0\lambda + 1 - \lambda_0.\end{aligned}$$

The Jacobian row $j = \partial\hat{y}/\partial w$ comes from `net_grad_point` with a unit seed on the observed slot, so **any carried derivative can be the observable**, Hessian components included.

The covariance P is quadratic in the number of weights. That is the method's intended regime: a small network updated very few times per pattern (the usual trade against SGD/Adam).

Element	Role
<code>kalman_t</code> <code>kf_num_weights(nt)</code> <code>kf_init(kf,nt,P0,lam,lam0)</code> <code>kf_update(kf,nt,tw,g,x,slot,y)</code> <code>kf_update_obs(kf,nt,j,xi)</code> <code>kf_free</code>	the filter type (P , forgetting factor λ , λ_0) the state dimension, for sizing P construction; P_0 is the diagonal initial value update on the observation of slot <code>slot</code> at point <code>x</code> with value <code>y</code> the form taking the Jacobian row and residual directly, for callers that assemble their own chain rule release

As verification, `libverify_example` confirms that Kalman training on a linear network reproduces the ridge-regression normal-equation solution to machine precision (measured 5×10^{-13}).

4.6 committee_module: uncertainty by committee

Returns the spread of several networks' predictions at the same input as an uncertainty indicator. The intended use trains the members separately (for instance with different `Rand_seed`) and bundles them at evaluation.

Element	Role
<code>committee_t</code> <code>comm_init, comm_free</code> <code>comm_eval</code>	the committee type (a set of member references) construction and release mean and spread at one point

4.7 blas_wrap_module

A single entry, `bgemm`: the $\alpha = 1, \beta = 0$ matrix product behind an explicit interface. The library's work spaces are pointer components, whose elements cannot be sequence-associated to the assumed-size dummies of the implicit BLAS interface, so callers hand whole arrays to this wrapper. Built without `USE_BLAS` it falls back to reference loops.

4.8 The C bindings

`lib/c_api_module.f90` and `lib/dnnf90.h` provide 22 functions. They avoid the `.mod` incompatibility, so embeddings that cross toolchains use these.

Function group	Role
<code>dnnf90_tables_init_dense</code> , <code>dnnf90_tables_init_closure</code>	shared tables (dense / closure)
<code>dnnf90_tabset_dense</code> , <code>dnnf90_tabset_closure</code> , <code>dnnf90_tabset_nderiv</code>	table sets and carried-count query
<code>dnnf90_net_create</code> , <code>dnnf90_net_create_ts</code> , <code>dnnf90_net_set_layer</code> , <code>dnnf90_net_load</code> , <code>dnnf90_net_free</code>	network construction, layer setup, loading, release
<code>dnnf90_work_create</code> , <code>dnnf90_work_free</code> , <code>dnnf90_twork_create</code> , <code>dnnf90_twork_free</code>	work spaces (evaluation / adjoint)
<code>dnnf90_grad_create</code> , <code>dnnf90_grad_zero</code> , <code>dnnf90_grad_free</code>	gradient accumulators
<code>dnnf90_eval</code>	one-point evaluation
<code>dnnf90_grad_point</code>	one point's weight gradient from a seed
<code>dnnf90_adam_step</code>	one step
<code>dnnf90_nderiv</code> , <code>dnnf90_alpha</code>	carried-count and multi-index queries

C wrappers for the committee and the Kalman filter are not provided (Section 11).

5 Input specification (`input_nn.dat`)

5.1 Format conventions

- One key per line, `KEY value ... /`; everything after `/` is ignored as a remark.
- A leading `#` or `!` makes a comment line.
- Key names are case-insensitive, and so are the named values (`Task`, `Activation`, `Init_w`, `GD_method`, the `Loss_term` form names, the residual term names, the NGD schedule names).
- If a key appears more than once, **the later line wins**.
- Key order is free. The only exceptions are the keys that need `Nlayer` first (`Loss_term`, `Residual`, `Time_axis`); appearing before `Nlayer` they stop the run.
- A word not in the parser's branches is skipped with the warning `input WARNING: unrecognized keyword ignored:.` A retired key is *not* skipped: it stops the run and states its migration path.

5.2 Required keys and defaults

Only five keys are required: `Nlayer`, `Init_w`, `Loss_term`, `GD_method`, `GD_param`. High-order fitting or collocation adds `Hod_K` and `Residual`. Everything else has a default. The smallest input is:

```
Init_w      Glorot_norm
Nlayer      3 /
1 /
8 /
1 /
Loss_term   DATA  200  train.dat  1.d0 /
Epoch      200 /
GD_method   Adam
GD_param    0.01d0 0.9d0 0.999d0 1.d-8 0.d0 /
```

5.3 Network and training

Key	Default	Meaning
Nlayer L /	required	layer count; the L lines that follow give one width each. Layer 1 is the input dimension D_0 ; the last layer must be 1
Activation TANH /	TANH	hidden activation. Anything but TANH stops the run, since the derivative tables are built from the Bell polynomials of tanh (see Section 5.4 for the other tabulated activations)
Init_w NAME	required	Glorot_unif, Glorot_norm, He_unif, He_norm, LeCun_unif, LeCun_norm, Random_unif, Random_norm. The first six set the variance from the fan-in (or fan-in and fan-out). Random_* is a fixed width-independent range (± 0.05 , or $\sigma = 0.05$), under which deep networks lose both signal and gradient
Rand_seed n /	0 (clock)	random seed; always set it for reproducibility
Restart n /	0	1 resumes from the weights in nn_weight.dat; the optimizer state is also restored when gd_*.dat exists (Section 6.3)
Task TRAIN PREDICT COMMITTEE /	TRAIN	PREDICT reads nn_weight.dat, writes the output files and exits. COMMITTEE evaluates a set of trained networks and exits (Section 5.9). Neither produces history, checkpoints or optimizer logs
Epoch n /	1000	maximum epoch count
Num_batch n /	$\min(10, N_{\text{train}})$	minibatch size; outside $[1, N_{\text{train}}]$ the run stops
Shuffle n /	0	1 redraws the training points each epoch
Num_validation n /	0	points held out for validation; outside $[0, N_{\text{input}} - 1]$ the run stops. At 0 the validation cost is undefined and early stopping reads the training cost
Validation_cyc n /	1	validation cadence (epochs); the history holds one line per validation
P_max n /	effectively infinite	the early-stopping patience: the run stops after this many validations without improvement. By default there is no early stop
Conv x /	0 (off)	stop when the training cost falls below this
Checkpoint_cyc n /	1000	cadence of checkpoint_ep*.dat
Output_deriv n /	0	1 writes the model's first derivatives after training; in a value-fit-only run the dense $K = 1$ table is built automatically
Hod_check n /	0	1 runs the in-run self-check before training (Section 7)
Average_cyc_mpi n /	1	weight-averaging cadence under MPI
Shuffle_cyc_mpi n /	10^6	training-point redistribution cadence under MPI

5.4 Choosing the activation

The adjoint reaches order $K + 1$, so the activation must be C^{K+1} . Among functions that satisfy this, **how fast the high-order derivatives grow** differs greatly, and this directly controls the conditioning of the carried slots. Measuring $\max_a |\sigma^{(q)}(a)|$ over $a \in [-6, 6]$:

Activation	$q = 2$	$q = 4$	$q = 6$	$q = 8$	parity
TANH	0.77	4.0	50	1220	odd
ERF	0.96	4.4	36	454	odd
SIN	1.0	1.0	1.0	1.0	odd
BESSEL	0.50	0.375	0.31	0.273	even
BESSEL1	0.41	0.334	0.288	0.257	odd

SIN has $\sigma^{(q)}(a) = \sin(a + q\pi/2)$, bounded by one at every order. BESSEL (J_0) and BESSEL1 (J_1) have

derivatives given by the binomial combination

$$J_\nu^{(q)}(a) = \frac{(-1)^q}{2^q} \sum_{k=0}^q (-1)^k \binom{q}{k} J_{\nu+q-2k}(a)$$

and are likewise bounded, since $|J_n| \leq 1$ at every order. J_0 is even, so the layer loses the sign of its preactivation; J_1 is odd with the same boundedness.

Only the BESSEL family uses externally derived code: the four routines `jyna` etc. of Zhang and Jin live in `lib/bessel_module.f90` with their copyright notice preserved (see LICENSE). The original routines treat negative arguments as $x = 0$, so a wrapping layer applies the reflection $J_n(-x) = (-1)^n J_n(x)$ — preactivations are of course signed, and omitting this would make half the layer return the constant $J_0 = 1$.

For ERF, only the value uses the Fortran 2008 intrinsic ERF, and that single file is excluded from `make f95check`. The derivatives are the Hermite functions $\text{erf}^{(q)}(a) = \frac{2}{\sqrt{\pi}} (-1)^{q-1} H_{q-1}(a) e^{-a^2}$, carried by the three-term recursion $\psi_{n+1} = 2a\psi_n - 2n\psi_{n-1}$ (with $\psi_n = H_n e^{-a^2}$), so no bare polynomial is formed and nothing overflows.

The oscillatory activations (SIN, BESSEL, BESSEL1) can use `Init_w SIREN`: an initialization that places the preactivations inside one period, with `Init_w.omega` (default 30) as the frequency. The optimum depends on the input range; on the shipped example with inputs in $[-1, 1]$, 12 works and 30 does not.

5.5 Evaluating a high-order fit

A single coefficient of determination pooled over all carried slots is useless at large K . On the shipped $K = 7$ example (330 slots) the mean square of the targets is nearly flat across orders (0.04–0.07) while the predictions blow up at high order, so one number hides the structure.

`bench/post/hod_accuracy.py` reports two dimensionless quantities per order,

$$\text{rel}(p) = \frac{\|T_{\text{pred}} - y\|_p}{\|y\|_p}, \quad \text{amp}(p) = \frac{\|T_{\text{pred}}\|_p}{\|y\|_p},$$

where $\text{rel} \geq 1$ means the order carries no information and $\text{amp} \gg 1$ means it diverges. Rankings use the relative error in the loss norm, $\|T - y\|_\lambda / \|y\|_\lambda$.

Measured this way, the activations differ structurally at $K = 7$: TANH reaches $\text{amp} = 14.7$ at order seven — the prediction swings fifteen times the target — against 4.7 for SIN, 5.2 for BESSEL, 3.9 for BESSEL1.

What λ_p does. The shipped example uses $\lambda_p = 1/(p!)^2$, so order seven carries weight 4×10^{-8} : the high orders are barely optimized at all. Re-measuring at $\lambda_p = 1$ (equal weight across orders) changes the picture entirely.

Activation	rel(7)		rel. error, loss norm
	$\lambda_p = 1/(p!)^2$	$\lambda_p = 1$	$\lambda_p = 1$
TANH	14.57	1.05	1.010
SIN	4.33	0.45	0.377
BESSEL	5.04	0.31	0.221

Equal weighting tames the order-seven divergence for every activation, but TANH then sits at $\text{rel} \approx 1$ **at every order**, with amp collapsed to 0.05–0.47: asked to actually deliver high orders, the tanh network degenerates to the mean everywhere. BESSEL holds $\text{rel} = 0.12$ –0.31 and $\text{amp} \approx 0.9$ at every order.

“The high orders do not fit” is therefore not an activation problem alone but also a loss-design problem — λ_p may be throwing the high orders away. In settings that genuinely demand them, the boundedness of the activation’s high-order derivatives is decisive.

5.6 Loss terms and their data files

5.6.1 The form of the loss

The loss is a weighted sum of terms. One `Loss_term` line declares one term; terms are numbered in order of appearance (the number that appears in the output file names).

$$L = \sum_t w_t L_t, \quad L_t = \frac{1}{2} \sum_{n \in t} \begin{cases} (N(\mathbf{x}_n) - y_n)^2 & \text{DATA} \\ \sum_{\alpha} \lambda_{|\alpha|} (T^\alpha(\mathbf{x}_n) - y_{n,\alpha})^2 & \text{HOD_DATA} \\ R(\mathbf{x}_n)^2 & \text{COLLOCATION} \end{cases}$$

w_t is the fourth argument of the `Loss_term` line. $\lambda_{|\alpha|}$ is the **per-order** weight, and λ_0 multiplies the value itself ($\alpha = 0$): a run with a single `HOD_DATA` term already contains a value fit whenever $\lambda_0 > 0$. Conversely, $\lambda_0 = 0$ leaves the value unconstrained — a function whose derivatives fit but whose constant is free: on `zwork/hod_4d.k3`, switching λ_0 from 1 to 0 drops the value’s coefficient of determination from 0.985 to -17.1.

5.6.2 What each form requires

This table narrows down what a given problem needs.

Form	Required keys	Data columns	Use
DATA	none	$D_0 + 1$	supervised values, boundary conditions
HOD_DATA	Hod_K	$D_0 + N_\alpha$	supervised derivatives
COLLOCATION	Residual (Hod_K optional)	$D_0 + \text{extra columns}$	points where the equation is imposed

D_0 is the number of input variables (the first width after `Nlayer`); N_α the number of carried multi-indices (Section 5.6.4). Data files are read list-directed, so **records are numbers only** — comment lines are not skipped. A record may wrap across lines (tokens are counted). Lines beyond the `n` of `Loss_term` are not read.

5.6.3 Worked examples

A value fit (DATA). $D_0 = 1$, $y = x^2 + 1$ at 200 points; one record is the two numbers x and y .

```

--- input_nn.dat ---
Nlayer      4 /
1 /  <- D0 = 1
12 /
12 /
1 /
Loss_term DATA 200 train.dat 1.d0 /

--- train.dat (200 lines) ---
-0.99004975124378  1.98019850993787
-0.98009950248756  1.96059503477637
...

```

A derivative fit (HOD_DATA). $D_0 = 4$, $K = 3$, dense set, so $N_\alpha = 35$. One record is $4 + 35 = 39$ numbers: x first, then the 35 targets in multi-index order. Note that the $K + 1$ lines directly after `Hod_K` are $\lambda_0, \dots, \lambda_K$ (they are continuation lines belonging to `Hod_K`; no other key may be interposed).

```

--- input_nn.dat ---
Nlayer      4 /
4 /  <- D0 = 4
10 /
10 /
1 /
Hod_K       3 /      <- K = 3 ; the next K+1 = 4 lines are the lambdas
1.d0        <- lambda_0 : multiplies the value
1.d0        <- lambda_1 : multiplies the first derivatives
1.d0        <- lambda_2
1.d0        <- lambda_3
Hod_dense   1 /      <- the dense set |alpha| <= K
Loss_term HOD_DATA 400 train.dat 1.d0 /

--- train.dat (400 records, 39 numbers each) ---
0.989957667989E+00 -0.774791878492E-01 -0.168822326093E+00 0.269057713255E+00 ...
|<----- x(1:4) ----->|<-- y_alpha(1:35) ...

```

Points where the equation is imposed (COLLOCATION). $D_0 = 2$. The residual contains `SRC` and `Exact_solution` 1 is set, so one record is x (2 numbers), the source f , and the exact solution u^* , **in that order** — 4 numbers. The exact solution is for reporting only; it does not enter the loss.

```

--- input_nn.dat ---
Loss_term DATA      160 data.dat      1.d0 /  <- boundary conditions

```

```

Loss_term COLLOCATION 400 colloc.dat 1.d-3 / <- interior points
Residual 3 /
LIN 1.d0 2 0 /
LIN 1.d0 0 2 /
SRC -1.d0 / <- declares that colloc.dat has an f column
Exact_solution 1 / <- declares that colloc.dat has a u* column

--- colloc.dat (400 records, 4 numbers each) ---
6.91978764042226e-01 9.36279743968240e-01 -3.23261059769365e+00 1.63765966006543e-01
|<----- x(1:2) ----->|<---- f(x) ---->|<---- u*(x) ---->|

--- data.dat (160 records, 3 numbers each: DATA, so x and y) ---
1.25000000000000e-02 0.00000000000000e+00 0.00000000000000e+00
|<----- x(1:2) ----->|<--- y = 0 --->|

```

The extra columns exist **only when declared**. Without SRC there is no f column; with Exact_solution 0 (the default) no u^* column. The smallest collocation file is the D_0 columns of $\mathbf{x}$ alone.

5.6.4 The carried multi-index set and N_α

Writing a HOD_DATA file requires knowing N_α and the **column order** first. The set is decided as follows.

Condition	Carried set	N_α
Hod_dense 1	all $ \alpha \leq K$	$\binom{D_0+K}{K}$
Hod_alpha_file given	the downward closure of the seed set	decided at run time
COLLOCATION only	the closure of the residual's multi-indices	decided at run time

For the dense set, N_α is computable in advance ($D_0 = 4, K = 3$ gives $\binom{7}{3} = 35$). The order is **ascending** in $|\alpha|$, lexicographic within an order. Every run writes `hod_alpha_order.dat`, which is the definition of the column order.

```

--- hod_alpha_order.dat (D0 = 4, K = 3) ---
# ia |alpha| alpha(1:D0)
1 0 0 0 0 0 <- the value
2 1 1 0 0 0 <- d/dx1
3 1 0 1 0 0 <- d/dx2
4 1 0 0 1 0
5 1 0 0 0 1
6 2 2 0 0 0 <- d^2/dx1^2
...

```

The theory of closures. A multi-index set $\mathcal{A}$ is **downward closed** when $\alpha \in \mathcal{A}$ and $\beta \leq \alpha$ (componentwise) imply $\beta \in \mathcal{A}$. The carried set must be closed because of the recursion's structure: the Bell part of Section 2.2 builds T^α from S^β and $B_{\alpha-\beta, q-1}$ with $\beta \leq \alpha$, and the adjoint reads $h_{\alpha-\beta}$; an index whose sub-indices are absent cannot be computed at all.

The closure of a seed set $\mathcal{S}$ is

$$\text{cl}(\mathcal{S}) = \bigcup_{\alpha \in \mathcal{S}} \text{box}(\alpha), \quad \text{box}(\alpha) = \{\beta : \beta \leq \alpha\}, \quad |\text{box}(\alpha)| = \prod_{i=1}^{D_0} (\alpha_i + 1),$$

a union of boxes, so $|\text{cl}(\mathcal{S})| \leq \sum_{\alpha} \prod_i (\alpha_i + 1)$ with equality for disjoint boxes; the exact count follows by inclusion-exclusion.

Take the shipped `zwork/hod_4dk7_active` ($D_0 = 4$). The seeds are $(7, 0, 0, 0)$, $(0, 0, 0, 1)$, $(2, 2, 1, 0)$, with box sizes 8, 2, $3 \cdot 3 \cdot 2 = 18$. The intersection $\text{box}(7, 0, 0, 0) \cap \text{box}(2, 2, 1, 0) = \{(k, 0, 0, 0) : k \leq 2\}$ has 3 elements; the other pairwise and the triple intersections are all $\{\mathbf{0}\}$ with 1. Hence

$$|\text{cl}(\mathcal{S})| = 8 + 2 + 18 - 3 - 1 - 1 + 1 = 24.$$

The run reports exactly $N_\alpha = 24$ — under 7% of the dense $\binom{4+7}{7} = 330$. Since the cost is $O(L n^2 N_\alpha)$, that ratio is exactly the gain in speed and memory a closure buys.

In a COLLOCATION-only run, no seed file is needed: **the residual's multi-indices are the seeds automatically**. The `bench/kdv` residual contains $u_t, u u_x, u_{xxx}$, i.e. $(0, 1)$, $(1, 0)$, $(3, 0)$, whose closure is the five indices $\{(0, 0), (1, 0), (2, 0), (3, 0), (0, 1)\}$ — half the dense $\binom{5}{3} = 10$, as the run also reports.

Ordering. The column order is ascending order of $|\alpha|$, and within an order it is the generation order of non-decreasing axis combinations (with repetition): choosing axis i once adds one to α_i , so at order 2 with $D_0 = 4$ the tuples $(1, 1), (1, 2), (1, 3), (1, 4), (2, 2), \dots$ produce $\alpha = (2, 0, 0, 0), (1, 1, 0, 0), (1, 0, 1, 0), (1, 0, 0, 1), (0, 2, 0, 0), \dots$. A closure **keeps that generation order and drops the absent indices**, so a closure preserves the dense set's relative order. Indexing is a binary search on the lexicographic order (`find_alpha`), which this ordering underwrites.

Getting the column order without a run. `tools/alpha_order.py` reproduces the same enumeration, byte for byte against `hod_alpha_order.dat`, so data files can be prepared before any run.

```
# the dense set
tools/alpha_order.py --d0 4 --k 3

# a closure (a seed file in the Hod_alpha_file format)
tools/alpha_order.py --d0 4 --seeds zwork/hod_4d_k7_active/alpha_seeds.dat

# just the count
tools/alpha_order.py --d0 10 --k 3 --count      -> 286
```

The three shipped variants (dense $D_0=4, K=3$, a seed closure, a residual-derived closure) are verified byte-identical against the run's `hod_alpha_order.dat`. Running once to obtain the order therefore still works, but is not necessary.

A record with the wrong column count is caught at read time; the run stops stating the measured and the required counts:

```
read_data: record 1 of train.dat
  accumulates 59 values where ndim(1)+NUM_alpha = 39 are required:
  a line is missing a value
  (x(1:ndim(1)) then y_alpha for every carried multi-index)
```

Because tokens are counted, a short record **eats into the next line** before the mismatch is noticed; the “measured” number is the accumulated count after the boundary slipped, not the token count of one line.

5.6.5 Key list

Key	Meaning
Loss_term FORM n file w /	declare one loss term. FORM is DATA, HOD_DATA or COLLOCATION; n the record count to read, file the data file, w the term weight ($w \geq 0$; negative stops). At most 100 terms
Hod_K K /	the maximum carried order K . The next $K + 1$ lines hold $\lambda_0, \dots, \lambda_K$, one per line. Required for HOD_DATA; optional for COLLOCATION-only runs, where the residual's highest order becomes K . An explicit K smaller than the residual's order stops the run
Hod_dense n /	1 forces the dense set $ \alpha \leq K$. At 0 (default) the closure is built from the Hod_alpha_file seeds and the residual's multi-indices; however, a HOD_DATA term without Hod_alpha_file implies the dense set, since the data columns correspond to all multi-indices
Hod_alpha_file f /	the seed multi-index file: first line the count, then one $\alpha(1:D_0)$ per line

5.7 Residuals (collocation)

Key	Meaning
Residual m /	define the residual operator in the next m lines, each one of the four kinds below. At most 64 terms after expansion
LIN c a1..aD /	cT^α (a linear term)
UUX c a1..aD /	cuT^α (cu^2 when $\alpha = 0$)

Key	Meaning
	<code>DXLAP c k [ix] /</code> $c \partial_{ix} \nabla^{2k} u$. The Laplacian runs over every non-time axis and expands multinomially into several linear terms. <code>ix</code> defaults to axis 1; a time-axis <code>ix</code> stops the run. At most 16 <code>DXLAP</code> lines
	<code>SRC c /</code> $c f(\mathbf{x})$ (the source, read per collocation point; at most once)
<code>Time_axis n /</code>	the index of the time axis; default the last axis, and every shipped case follows that convention. <code>DXLAP</code> expands after the whole input is read, so this key may appear anywhere
<code>Exact_solution n /</code>	1 reads the collocation file's last column as the exact solution and adds the error to the output; it never enters the loss

The residual is evaluated pointwise as $R = \sum_k c_k [u] T^{\alpha_k} + c_{\text{src}} f$, and the seed is $\partial L / \partial T^{\alpha} = w R \partial R / \partial T^{\alpha}$; the source is additive and does not appear in $\partial R / \partial T^{\alpha}$. Residual evaluation and seed construction are argument-only functions with no shared state.

A collocation run writes the parsed residual back to the log head in human-readable form. The operator is the one quantity in the input with nothing to cross-check against — a mistyped multi-index just solves a different equation correctly — so this echo is the only defence:

```
### Residual as parsed (imposed as R = 0 at every collocation point)
### axes are written x1..x2, with axis 2 shown as t (Time_axis)
### term      coefficient  expression      slot  multi-index
### 1         1.000000E+00  u_t            3     0  1
### 2         3.000000E+00  u * u_x1       2     1  0
### 3         1.000000E+00  u_x1x1x1       5     3  0
```

The last column is the slot of the carried multi-index, checkable against `hod.alpha.order.dat`.

5.8 Coupled systems (the System block)

The scalar `Residual` block writes one residual of one field. **Systems of residuals over several field components** — incompressible Navier–Stokes, or a charge density coupled to a potential — are written with the `System` block. The output width m becomes the number of field components (the scalar form's width-1 constraint is lifted when `System` is present), and training carries every derivative of every component to order K .

Each term is a product of at most four derivative factors,

$$c \partial^{\gamma} u_k \partial^{\beta} u_i \partial^{\alpha} u_j,$$

and the four term kinds are its cases: `TRM` one factor, `XUX` two with the first undifferentiated ($\beta = 0$), `DXD` two differentiated factors, `TRP` three. With one component and `ic = jc = 1` they reduce to `LIN/UUX`, so existing inputs keep their meaning.

Key	Meaning
<code>System nres nterm /</code>	define <code>nres</code> residuals (≤ 16) in the next <code>nterm</code> lines (≤ 256). <code>ir</code> is the residual a term belongs to; <code>jc</code> , <code>ic</code> , <code>kc</code> are component indices
<code>TRM ir jc a1..aD c /</code>	$c \partial^{\alpha} u_{jc}$ (linear)
<code>XUX ir ic jc a1..aD c /</code>	$c u_{ic} \partial^{\alpha} u_{jc}$: advection $u u_x + v u_y$, charge transport by a flow, the body force ρE — the terms coupled systems need and the scalar form cannot write
<code>DXD ir ic b1..bD jc a1..aD c /</code>	$c \partial^{\beta} u_{ic} \partial^{\alpha} u_{jc}$ (a product of derivatives). Flux divergences need it: in $\nabla \cdot (\rho \nabla \phi) = \nabla \rho \cdot \nabla \phi + \rho \nabla^2 \phi$ the first term is a product of first derivatives of different components
<code>TRP ir kc g1..gD ic b1..bD jc a1..aD c /</code>	$c \partial^{\gamma} u_{kc} \partial^{\beta} u_{ic} \partial^{\alpha} u_{jc}$ (three factors), needed by the compressible momentum flux $\rho u u_x$ or field-dependent mobilities

Key	Meaning
QAD ir lc d1..dD kc g1..gD ic b1..bD jc a1..aD c /	$c \partial^\delta u_{lc} \partial^\gamma u_{kc} \partial^\beta u_{ic} \partial^\alpha u_{jc}$ (four factors), the quartic nonlinearity of the higher dispersive hierarchies: the seventh-order Lax equation carries $140 u^3 u_x$
Sys_src /	declares that the collocation file carries one source column per residual , between the coordinates and the exact solutions. Omit for a homogeneous system, whose file is that much narrower; a mistaken setting stops with a record-length error — the check working as intended
Sys_wcomp w1..wm /	per-component weights of the supervised loss (default all 1). Mandatory for components of different magnitudes: an unweighted sum of squares is a statement about the largest component only. The custom is to measure $1/\ y_i\ ^2$ from the data, as the shipped cases do
Sys_wres w1..wr /	per-residual weights of the collocation loss (default all 1), for the same reason. Reads <code>nres</code> values, so it must follow <code>System</code>
Sys_rnoise r1..rr /	per-residual observation noise for the extended Kalman filter. Exact, not approximate: scaling both the observation row and the innovation by $1/\sqrt{r}$ reproduces noise r on the original quantity exactly. A residual with an outsized source produces out-sized innovations, and a filter trusting all observations equally spends its covariance on that residual; raising its noise says “this observation is less trustworthy”. Must follow <code>System</code>
Sys_balance cyc alpha /	every <code>cyc</code> epochs, readjust <code>Sys_wres</code> so that each residual contributes comparably to the gradient. Each residual’s own gradient norm is sampled on up to 40 training points, and each weight moves a fraction α toward $g_{\max}/g_{ir}$ (the running average keeps it from chasing one batch’s noise). The weights are printed at each readjustment

The system data formats: `data.dat` is $\mathbf{x}(1:D_0) y(1:m)$; `colloc.dat` is $\mathbf{x}(1:D_0)$, then one source per residual if `Sys_src`, then all components’ exact solutions if `Exact_solution 1`. The reader counts values, so a column slip stops with a record-length error rather than propagating silently.

Residual evaluation (`calc_sys_residual`) and seed construction (`set_sys_seed`) are argument-only functions, like the scalar form. Each factor of a product contributes to the seed by the product rule; the adjoint of that is checked against finite differences for one to three factors by `tools/example_product_adj.f90` (9.5×10^{-12}), and for the four-factor form by the in-run check of `zwork/pinn_lax7` (2.0×10^{-10}), and the multi-output forward and adjoint by `tools/example_multiout*.f90` (2.2×10^{-10} , 9.1×10^{-12}).

The shipped examples are `zwork/pinn_lax7` (the seventh-order Lax equation, whose quartic term needs the four-factor form; exact solitary wave, table read back to 1.1×10^{-14}), `zwork/pinn_lax` (Lax’s fifth-order equation: a scalar case whose terms $30u^2u_x$ and $20u_xu_{xx}$ need the product language; exact soliton, term table read back independently to 8×10^{-16}), `zwork/pinn_kovaszny` (Kovaszny flow; exact solution, term table verified against an independent implementation to 10^{-15}), `zwork/pinn_cavity` (the $Re=100$ cavity, compared with the Ghia benchmark profiles), and `zwork/pinn_ehd` (a five-component electrohydrodynamic system; manufactured solution).

5.9 Uncertainty by committee

Task `COMMITTEE` uses `committee_module` (Section 4) from the trainer. Members are independently trained networks and cannot be produced in one run (usually the same input is trained under different `Rand_seed` and each `nn_weight.dat` saved under its own name). This task reads them and writes, for every input point, **the mean and sample standard deviation of every carried slot**, then exits. The point is that the spread of the *derivatives* is available, not just of the value: with `Output_deriv 1` the dense first-order set

is carried, so the spread of $\partial N/\partial x_i$ is reported too — exactly the quantity an on-the-fly scheme thresholds when deciding whether to call the expensive first-principles calculation.

Key	Meaning
<code>Committee m /</code>	the next m lines name the members' weight files, one per line, $m \geq 2$. Members must share the multi-index set the current input defines; a slot-count mismatch stops the run

The output is `output_committee.set<4 digits>.dat`: one line is $\mathbf{x}(1:D_0)$, all slots' means, all slots' standard deviations. At the end the spread over the whole input set is summarized per slot (mean and maximum). The shipped example is `zwork/tour_committee` (`train_members.sh` trains four members; `committee.dat` evaluates them).

5.10 Optimizers

Key	Meaning
<code>GD_method NAME</code>	the eleven gradient rules <code>SIMPLE</code> , <code>SIMPLE_SCHEDULE</code> , <code>SIMPLE_GMAXCLIP</code> , <code>MOMENTUM</code> , <code>NESTEROV</code> , <code>ADAGRAD</code> , <code>ADADELTA</code> , <code>RMSPROP</code> , <code>RMSPROP_NESTEROV</code> , <code>ADAM</code> , <code>NATURAL_GRAD</code> ; the non-gradient <code>KALMAN</code> ; and the quasi-Newton <code>LBFGS</code>
<code>GD_param p1 ... p5 /</code>	the method's hyperparameters. An independent key, in either order with <code>GD_method</code> . All five zero stops the run, since no method can take a step (only <code>LBFGS</code> , whose line search sets the step, is exempt and may omit <code>GD_param</code>). The adaptive rules (p_2 for <code>ADAGRAD</code> , <code>ADADELTA</code> ; p_3 for <code>RMSPROP</code> , <code>RMSPROP_NESTEROV</code> ; p_4 for <code>ADAM</code>) divide by an accumulator regularized by ε , so a non-positive slot stops the run (a weight with zero first-step gradient would divide by zero)

p_1 is the learning rate for every rule. `SIMPLE_SCHEDULE` uses $\eta(t) = p_1/\sqrt{t+1} + p_2$; `MOMENTUM` and `NESTEROV` take p_2 as the momentum; `ADAM` uses p_2, p_3, p_4 as $\beta_1, \beta_2, \varepsilon$.

`KALMAN` selects the per-pattern extended Kalman filter of `kalman_module` (Section 4) from the trainer. Rather than accumulating a gradient and stepping once, it presents scalar observations one at a time and lets the rank-one update move the weights itself. The observable per loss form: the value for `DATA`; each carried derivative with $\lambda_{|\alpha|} \neq 0$ for `HOD_DATA`; the residual (target 0) for `COLLOCATION`. The Jacobian row $\partial \hat{y}/\partial \mathbf{w}$ comes from the same seed path the gradient rules use. `GD_param` means: p_1 = the diagonal initial value of $\mathbf{P}$; p_2 = the initial forgetting factor λ ; p_3 = the λ_0 of $\lambda \leftarrow \lambda_0 \lambda + 1 - \lambda_0$; the run stops unless $p_1 > 0$ and $0 < p_2, p_3 \leq 1$. $\mathbf{P}$ is quadratic in the weight count, so this is for small networks; the run reports the size at startup and refuses above 2 GB.

`Kalman_gate g /` is the innovation gate (a robust EKF): an observation with $\xi^2 > g^2(\lambda + \mathbf{j}^\top \mathbf{P} \mathbf{j})$ has its row and innovation shrunk by the same factor (the same exact mechanism as `Sys_noise`) until equality — every rank-one step becomes bounded, at the price of absorbing surprising observations more slowly. Default 0 (off).

The forgetting factor needs care. $\lambda < 1$ re-inflates $\mathbf{P}$ by $1/\lambda$ at every update, so under thousands of updates per epoch the gains never decay and the filter can drift off a fitted state along the linearization error of a nonlinear residual (observed on coupled systems with source terms). With $p_2 = p_3 = 1$ (λ pinned at one) the gains decay as in recursive least squares and the filter holds and refines a fitted state (a gentle descent at $p_1 = 10^{-5}$). Forgetting buys initial convergence speed at the cost of having no stationary behaviour: use forgetting for cold starts, and $\lambda = 1$ with small p_1 to refine a fitted state. `Kalman_iter n /` is the iterated EKF (IEKF): each observation is relinearized n times at the running iterate before the covariance is updated once (Gauss–Newton on the single-observation MAP problem). The measurement on the coupled EHD system is negative and worth recording: neutral in the stable regime, and it **amplifies** the divergence in the forgetting regime (0.44 against 0.023 from the fitted start). Solving thousands of mutually contradictory scalar observations each more exactly in sequence strengthens the oscillation between them; the removed linearization error had been acting as a buffer. The collective cure for this failure mode is a simultaneous Gauss–Newton over all observations — that is, full-batch quasi-Newton. `Kalman_q q /` adds q to the diagonal of $\mathbf{P}$ after each update: process-noise injection, a stationary behaviour intermediate between forgetting (inflating everything) and $\lambda = 1$ (gains vanishing). Default 0. `Kalman_mode DECOUPLED /`

replaces the covariance with the node-wise block diagonal (bias + fan-in as one block per neuron, Puskorius–Feldkamp). The gain denominator $\lambda + \mathbf{j}^\top \mathbf{P} \mathbf{j}$ stays global; only the cross-neuron covariance is dropped. Memory and update cost fall from n_w^2 to the sum of squared block sizes (measured $87\times$ per epoch on the shipped EHD net), the 2 GB dense guard no longer applies, and force-field widths become reachable. The price is the convergence rate: on the KdV bench 1.2×10^{-7} in 0.5 s where the dense filter reaches 7×10^{-14} in about 6 s; on Poisson 2D 3.2×10^{-8} in 5.4 s / 3000 epochs against 5.5×10^{-10} in 6.5 s / 300 — roughly ten times the epochs per decade at a tenth the cost per epoch.

The shipped pair `zwork/pinn_poisson2d` and `zwork/pinn_poisson2d_kalman` differ only in the method (same problem, data, network) and measure, against the exact solution:

Method	Epochs	Time	Training cost	RMSE
zwork/pinn_poisson2d (2-D Poisson: linear operator + source)				
Adam	20000	60 s	1.1×10^{-2}	3.1×10^{-3}
Kalman	300	8 s	4.8×10^{-9}	1.8×10^{-5}
bench/opt_* (KdV, with the nonlinear term Auu_x)				
Simple	20000	6 s	2.4×10^{-3}	—
Natural_grad	3000	36 s	1.5×10^{-5}	—
Kalman	1000	6 s	2.6×10^{-13}	3.8×10^{-7}

The KdV residual contains Auu_x and is not a linear functional of the carried derivatives. The filter then receives the residual value directly through `kf_update_resid` (building the prediction as `dot(seed, T)` via `kf_update_obs` would double-count the quadratic term), so `bench/opt_kalman` also verifies the observation path for nonlinear operators.

Only the natural gradient carries further keys. The metric is $G = \frac{1}{N_b} [\sum_n \mathbf{j}_n \mathbf{j}_n^\top + \mu \text{tr}(\cdot) I]$ and the update $\Delta W = -\eta(t) G^{-1} (\nabla / N_b)$. The rows $\mathbf{j}_n$ are $\partial u / \partial W$ for DATA terms (a Gauss–Newton metric) and $\partial R / \partial W$ for COLLOCATION terms; for HOD_DATA terms the per-point loss gradient (an empirical Fisher metric).

LBFGS is the limited-memory quasi-Newton method: the search direction comes from the two-loop recursion over the most recent pairs $(\mathbf{s}, \mathbf{y})$ (verified by `tools/example_lbfgs_check.f90`, 1.3×10^{-16}), and the step from a backtracking line search on the Armijo condition $L(\mathbf{w} + t\mathbf{d}) \leq L(\mathbf{w}) + c_1 t \mathbf{g} \cdot \mathbf{d}$. There is no learning rate. It is a **full-batch method**: a pair $(\mathbf{s}, \mathbf{y})$ is meaningful only when consecutive gradients belong to the same function, and a redrawn minibatch swaps the function at every step; `Num_batch` must therefore equal the training count (all points minus `Num_validation`), and the input check stops with the required value on violation. `Lbfgs_ss` / is self-scaled BFGS (SSBFGS, Oren–Luenberger): instead of applying the newest pair’s scale $\mathbf{s} \cdot \mathbf{y} / \mathbf{y} \cdot \mathbf{y}$ once to the initial Hessian, every stored pair receives its own factor $\tau_j = \mathbf{s}_j \cdot \mathbf{y}_j / (\mathbf{y}_j^\top H_j \mathbf{y}_j)$ at its own level of the recursion. H_j is the operator built from the older pairs, so the factors are fixed oldest-first by a nested application of the same recursion ($O(m^2 n_w)$ per direction, negligible beside one full-batch gradient). With a single stored pair it coincides exactly with plain L-BFGS (measured: eight digits over fifty iterations). The measured verdict is stated honestly: on the problems of this distribution it is slightly worse than plain L-BFGS (5.2×10^{-3} against 3.3×10^{-3} at 300 cold EHD epochs). `Lbfgs_wolfe c2` / adds the curvature (second Wolfe) condition $\mathbf{g}(\mathbf{w} + t\mathbf{d}) \cdot \mathbf{d} \geq c_2 \mathbf{g}(\mathbf{w}) \cdot \mathbf{d}$ ($c_1 < c_2 < 1$) to the line search: a step that lowers the loss while the slope is still steep is rejected and the step is grown rather than shrunk (with proper bracketing; each such trial costs a gradient instead of a cost evaluation). The default 0 reproduces the pure Armijo search exactly. On the problems of this distribution it changes nothing — the first trial $t = 1$ already satisfies both conditions, so the search was not the limiter; the measurement is recorded with the feature.

`Lbfgs_expand n` / is forward tracking: when the first trial satisfies Armijo, the step is doubled up to n times and the longest step that still satisfies the condition and lowers the loss is taken (default 0 reproduces pure backtracking exactly). Measured on the EHD cold start it gives a 19 % lower loss at equal epochs, but loses at equal wall time due to the extra cost evaluations. The memory size is `Lbfgs_m n` / (default 8, range [1, 200]). More memory improves the per-epoch quadratic model (the two-loop’s extra cost is $O(n n_w)$): on the five-component EHD cold start the objective at 300 epochs measures 6.3×10^{-3} at $m=8$ against 3.3×10^{-3} at $m=40$. That case admits **one-shot training from random weights** (`zwork/pinn_ehd/input_cold.dat`: objective 3.8×10^{-4} in 900 epochs, about 250 s, $R^2 \geq 0.9975$ in all components). If no loss-lowering step is found, the memory is discarded and the run proceeds to the next epoch (`Lbfgs_scan` / enables extra backtracking before giving up; `Lbfgs_verbose` / prints the accepted and the recomputed loss at each exit). The objective the line search itself minimizes is recorded in `lbfgs_history.dat` (one line = epoch, accepted loss, step, trial count, acceptance flag); the training history is a different sum common to every update rule and is left alone.

The natural gradient’s damping is chosen by `Ngd_damping`: **TRACE** is $G + \mu \text{tr}(G) I$ (relative; its meaning depends on the weight count), **ABS** is $G + \mu I$ (absolute; independent of it — the one that works at the network

sizes coupled systems need). Default `TRACE`. `Ngd_dual /` solves the natural-gradient step through the Gram (dual) matrix: the push-through identity

$$(G + cI)^{-1}b = \frac{1}{c} \left[b - \frac{1}{N} J^\top K^{-1} J b \right], \quad K = cI + \frac{1}{N} J J^\top$$

(with K of row dimension) yields **the identical step** to the dense $n_w \times n_w$ solve at $O(N^2 n_w + N^3)$ (verified against the primal route on the EHD case to every printed digit; 140 s/epoch primal against 0.16 s/epoch dual there). The dense 2 GB guard does not apply to the dual route.

For a system the metric carries **one row per residual** (one per component for a multi-component fit), built with the exact $\sqrt{w_r}$ factorization of the weighted objective. A single row per point ($d(\sum_r w_r R_r^2/2)/d\mathbf{w}$) would collapse the outer product to the rank-one-per-point empirical Fisher, blind to how the residuals trade against each other — the very difficulty of a coupled system. With this and the dual solve, the natural gradient solves the EHD system from random weights (minibatch of 40, $0.743 \rightarrow 2.8 \times 10^{-4}$ in 900 epochs, about 150 s, $R^2 \geq 0.9986$ in all components; `zwork/pinn_ehd/input_cold_ngd.dat`).

`Ngd_geo h alpha /` is geodesic acceleration (Transtrum–Sethna): the second directional derivative of every observable along the step δ_1 is measured by one extra evaluation, a second solve against **the same metric** (cheap in the dual route) gives the correction δ_2 , and $\delta_1 + \delta_2/2$ is taken when $\|\delta_2\|/2 \leq \alpha\|\delta_1\|$ (an η -consistent finite difference). Measured neutral: within trajectory noise on the problems of this distribution, at about twice the cost per epoch.

`Ngd_trust mu0 /` is a Levenberg–Marquardt rule adapting the absolute damping from the observed loss change: $\mu \leftarrow \mu/3$ on descent, $\mu \leftarrow 5\mu$ on ascent, bounded to $[10^{-6}, 10^8]$. Setting it forces `Ngd_damping ABS`.

Key	Meaning
<code>Ngd_schedule_eta NAME a b /</code>	the schedule of $\eta(t)$: <code>NONE</code> (constant p_1), <code>SIMPLE</code> ($p_1/(1+at)^b$), <code>EXP</code> ($p_1 e^{-at}$)
<code>Ngd_schedule_mu NAME a b /</code>	the schedule of $\mu(t)$; the same three
<code>Ngd_eta_bound x /</code>	lower bound on η
<code>Ngd_mu_bound x /</code>	lower bound on μ

6 Output files

Everything is written to the run directory; build products collect in `build/`.

6.1 The training history `history_ep<start epoch, 7 digits>.dat`

One line per validation. The file name contains the starting epoch, so a restarted run opens a new file rather than appending. The leading `#` lines are self-describing. Every metric is normalized **per point**.

Each metric has two channels: the **value channel** (`.f`) is the contribution of $N(\mathbf{x}) - y$; the **derivative/residual channel** (`.df`) that of the high-order terms and the collocation residuals. The total columns (3–5) are assembled per form: `DATA` value channel only, `COLLOCATION` residual channel only, `HOD_DATA` λ_0 -value + derivatives.

Columns	Content
1	epoch
2	the patience counter (compared with <code>P_max</code>)
3–5	total cost: training, validation, all input points
6–9	cost split: training (value, deriv), validation (value, deriv)
10–11	all-points cost split (value, deriv)
12–17	RMSE: training (value, deriv), validation (value, deriv), all points (value, deriv)
18–23	MAE, same layout
24–	only with two or more loss terms: 21 columns per term (the layout of 3–23), in order — 24–44 for term 1, 45–65 for term 2, and so on

A `DATA+COLLOCATION` run, for instance, has 65 columns per line, with the derivative-channel columns of term 1 (`DATA`) zero and the value-channel columns of term 2 (`COLLOCATION`) zero.

6.2 Predictions

File	One line holds
output_set<4 digits>.dat	$\mathbf{x}(1:D_0)$, prediction, target. One file per loss term, numbered in <code>Loss_term</code> order. For a <code>COLLOCATION</code> term the target column carries the exact solution (with <code>Exact_solution 1</code>)
output_hod_set<4 digits>.dat	$\mathbf{x}(1:D_0)$, all predicted carried derivatives $T^\alpha(1:N_\alpha)$, all targets. Written for <code>HOD_DATA</code> terms
output_deriv_set<4 digits>.dat	$\mathbf{x}(1:D_0)$, $\partial N/\partial x_i$ ($i = 1..D_0$). Only with <code>Output_deriv 1</code>

Since the last two columns are prediction and target, the quality of a fit is one line of arithmetic away. A coefficient of determination near one means the model explains the data; near zero it has learned nothing beyond the target mean. A model outputting a constant still produces smooth, plausible-looking files, so this check is worth doing every time.

6.3 Weights and restarting

File	Content
nn.weight.dat	the best weights (by training cost). Line 1 the best epoch; lines 2–3 reserved fields (activation codes, currently always 0); then the layer count, the widths, and per layer a <code>#1=</code> line followed by the rows $W_{j,0:n_l-1}^{(l)}$
checkpoint_ep<7 digits>.dat	the weights every <code>Checkpoint_cyc</code> ; same format, line 1 its own epoch. Copied over <code>nn.weight.dat</code> it becomes a restart point
gd.dw.dat, gd.m.dat, gd.v.dat, gd.r.dat, gd.u.dat	the optimizer state (previous step and the moments); how many exist depends on the method

`Restart 1` reads the weights from `nn.weight.dat`. If `gd.dw.dat` is present in the same directory, **the optimizer state is restored too**, so Adam resumes without losing its moments; absent, only the optimizer starts cold. If the epoch of the weight file and the optimizer log disagree (an old checkpoint copied in with the optimizer log left behind), the run stops.

6.4 Reference information

File	Content
hod_alpha_order.dat	the carried multi-index order: slot, $ \alpha $, $\alpha(1:D_0)$ per line. Defines the column order of high-order data files and the slots of the residual echo
data_division.log	the indices of the points held out for validation
hod_golden.dat	known values for the self-check; absent, the regression comparison is skipped

7 Verification specification

With a single propagation engine, correctness is checked **against the definition**: finite-difference verification of the weight gradient at three levels.

Level	Object	Measured (max. rel. error)
tools/dbg_backward_fd.f90 (point)	a synthetic dense net ($D_0 = 2$, $K = 3$), arbitrary seed	2.2×10^{-10}
same (batch vs. pointwise)	one batch along two paths	0 (loop build)
same (batch vs. FD)	central differences of the batch loss	1.4×10^{-11}

Level	Object	Measured (max. rel. error)
<code>train_example</code> test 1	trained weights, arbitrary seed	2.9×10^{-11}
<code>Hod_check</code> 1	in-run, real data and loss	3.4×10^{-10}

Finite differences bound the derivative error but are blind in principle to defects that leave the converged value intact — a write one element past an array, a read of a never-set variable, a division that is only sometimes by zero. That layer is covered by two means:

- **make harden**: a build with bounds checking, uninitialized values poisoned by a signalling NaN, and IEEE traps (invalid / zero / overflow). The shipped cases run under it with zero violations.
- **make negtests**: the input-rejection contract and exact properties. Malformed or incomplete inputs must stop with a diagnostic, and properties that hold exactly must do so (a committee of identical members returns exactly zero spread, among others).

The checks run under both the loop build and the `BLAS=1` build, which compile different kernels (explicit loops against matrix products). The two compute the same quantities in different summation orders: the gradients agree to rounding, and four of the five benchmarks have bit-identical histories, but a long run can fork once a last-digit difference appears (`kdv` is bit-identical for 60 validation cycles, then forks to a final cost of the same size). Numbers quoted from runs therefore state which build produced them. Finite differences are judged relative to the gradient norm: central differences cannot resolve components orders of magnitude below the largest, so per-component ratios would measure the reference’s noise rather than the adjoint’s error.

The self-check’s pass criterion is **relative to the size of the gradient itself**. Dividing by $\max(|fd|, 1)$ becomes an absolute tolerance once the gradient drops below one — on a loss of order 10^{-3} it would test only three digits. And the known-value check fails on zero comparisons: a `hod_golden.dat` generated with a different D_0 , K or closure would have every row skipped as out of range and could otherwise pass with nothing compared.

The self-check `Hod_check` 1 sets deterministic weights and verifies (a) the carried derivatives against central differences in x (orders 1–2 only; at $K > 2$ the higher slots are covered through the seed path by (b) and by `fdcheck`), (b) the weight gradient against central differences of the loss, (c) regression agreement with the known values (`hod_golden.dat`); any miss prints **FAILED**. What is checked is the exact path that ships.

As a regression net, the training-history columns of five benchmarks (`kdv`, `zk7`, `g7`, `opt_ngd`, `opt_kalman`) are held as frozen references. Thread against serial accumulation (3.2×10^{-15}), the concurrent training of several networks, and the agreement of Kalman training with the ridge-regression solution (5×10^{-13}) are checked by `train_example` and `libverify_example`.

8 Application to force fields

`tools/example_hod_ff.f90` demonstrates a force field trained on energies, forces and **analytic Hessians**. Descriptor pipelines (`n2p2` and `kin`) provide dG/dR but not d^2G/dR^2 , so they can train up to forces but not Hessians; this gap is the use that states the library’s position most clearly.

The model is a sum of atomic energies $E = \sum_i N(G^{(i)})$, carrying the dense set at $D_0 = 3$, $K = 2$ ($N_\alpha = 10$). The model Hessian is assembled by the chain rule,

$$H_{pq} = \sum_i \left[\sum_{ab} \frac{\partial^2 N}{\partial G_a \partial G_b} \frac{\partial G_a^{(i)}}{\partial x_p} \frac{\partial G_b^{(i)}}{\partial x_q} + \sum_a \frac{\partial N}{\partial G_a} \frac{\partial^2 G_a^{(i)}}{\partial x_p \partial x_q} \right],$$

where $\partial^2 N / \partial G_a \partial G_b$ of the first term is the carried slot $\alpha = e_a + e_b$ and $\partial N / \partial G_a$ of the second is $\alpha = e_a$. The loss seeds land on the same slots, so **Hessian training costs one ordinary adjoint per atom and configuration**.

8.1 Verification

The demonstration contains three hand-written derivative assemblies (the descriptor derivatives, the model Hessian, the seed of the Hessian loss). Any of them would train a *plausible* force field with one wrong coefficient, so each is checked by central differences before any physics is reported. Measured (relative):

Check	Max. relative difference
dG/dx , d^2G/dx^2 vs. differences of G	4.7×10^{-10}
model Hessian vs. differences of the model energy	4.4×10^{-7}
dL/dw with the Hessian term vs. differences of L	6.1×10^{-8}

8.2 The smoothness requirement (important)

Because second derivatives are propagated, **the cutoff function must be C^2 as well**. The cosine cutoff $f_c = (\cos(\pi r/r_c) + 1)/2$ used by the demonstration has $f_c(r_c) = f'_c(r_c) = 0$ but $f''_c(r_c) = +\frac{1}{2}(\pi/r_c)^2 \neq 0$: the second derivative jumps at r_c , so a Hessian-trained model is only C^1 across the cutoff.

The demonstration works because no pair of the sampled configurations approaches r_c — a property of the sample, not of the method. The program therefore **checks that margin rather than assuming it** (measured 0.24 in equilibrium-spacing units). A production coupling should use a C^2 cutoff (a polynomial, or $\tanh^3$).

This is the same general rule stated for the activations: **the smoothness class of every ingredient must exceed the order of the derivatives one intends to use**.

8.3 Result

Two networks are trained from the same initial weights on the same 60 configurations under the same schedule:

Training data	Max. rel. error, force constants	Max. rel. error, phonons
E , F (a descriptor pipeline’s reach)	1.85%	3.84%
E , F , H (this library)	0.55%	1.12%

The figure is drawn from the run logs by `bench/post/make_fig_forcefield.py`.

9 Shipped examples and test cases

Where	Content
<code>bench/</code>	the numerical experiments of the methods paper: KdV, Kawahara, Zakharov–Kuznetsov (orders 3/5/7), Gross–Pitaevskii, the slit, the 10-dimensional EYU case, closure/dense scaling, optimizer comparison, speed comparison
<code>zwork/</code>	the keyword tour: a value fit (<code>x2+1</code>), high-order fits (<code>hod_4d_k3</code> , <code>hod_4d_k7</code>), collocation (<code>pinn_kdv</code> , <code>pinn_zk7</code>), coupled flow systems and System cases (<code>pinn_lax</code> , <code>pinn_kovasznyay</code> , <code>pinn_cavity</code> , <code>pinn_ehd</code>), a derivative fit (<code>tour_derivfit</code>), a Sobolev loss (<code>tour_sobolev</code>), learning-rate schedules (<code>tour_schedule</code>)
<code>tools/</code>	worked examples: embedding (<code>example_embed</code>), force fields (<code>example_mlff</code>), the training API (<code>example_train</code>), verification (<code>example_libverify</code>), committees (<code>example_uq</code>), custom losses (<code>example_customloss</code>), the C examples
<code>host/</code>	use from a host code

10 Cost and memory

With N_α multi-indices and width n , one point’s forward pass costs $O(L n^2 N_\alpha)$ in the linear part and $O(L n (\text{table entries}))$ in the Bell part; the reverse pass is of the same order. The work space is $O(L n N_\alpha)$ (the adjoint plane covers one layer only). N_α is $\binom{D_0+K}{K}$ for the dense set and falls to the necessary minimum under a closure.

A value-fit-only run takes the batch path (`net_grad_batch`), where the whole minibatch becomes three matrix products per layer (under `USE_BLAS`).

11 Known limitations

- The hidden activation is $\tanh$ (plus the tabulated alternatives of Section 5.4); carrying another activation needs a generator for its derivative table.
- The read-out is linear only. The scalar forms (`Residual`, `HOD_DATA`) have one output; the `System` block (Section 5.8) sets the output width to the number of field components and carries the seed as a (component $\times$ multi-index) matrix (verified by `tools/example_multiout*.f90`).
- No residual connections, convolutions or attention (fully connected only).
- The primal natural gradient builds a dense metric and is for small networks (with a stop on quadratic memory); the dual route (`Ngd_dual`) removes that limit for minibatch row counts.
- `checkpoint_ep*.dat` holds weights only; the optimizer state lives in `gd_*.dat`. Restarting from an old checkpoint therefore requires the matching optimizer log (a mismatch is detected and stops the run).
- The scalar residual's nonlinearity is limited to uT^α . The `System` block writes products of up to three derivative factors (`XUX`, `DXD`, `TRP`), so $\nabla\rho\cdot\nabla\phi$ and ρuu_x are expressible; transcendental functions of the field, $\sin u$ or e^u , are expressible in neither form.
- Residual coefficients are constants. Space-dependent coefficients $c(\mathbf{x})$ (inhomogeneous media, potential terms) are not expressible; since `SRC` already reads a per-point value, they would extend that mechanism.
- There is no bare-Laplacian keyword: $\nabla^2 u$ is written as one `LIN` line per dimension (`DXLAP` is $\partial_x \nabla^{2k} u$ specifically).
- C wrappers for the committee and the Kalman filter are not provided.